

Property Peace Information Security and Payment Operations Policy

Policy version: 1.0 Draft

Prepared: July 30, 2026

Classification: Internal / Confidential

Policy owner: Business Owner / CEO (to be formally assigned)

Security owner: To be assigned

Payment Operations owner: To be assigned

Approval status: Draft — pending owner approval

Effective date: Upon written approval

Review cycle: At least annually and after every material security or payment incident

1. Purpose

This policy establishes mandatory business, information-security, software-development, payment-processing, incident-response, and continuity controls for Property Peace. Its purpose is to protect customers, tenant and landlord information, company systems, and company funds while maintaining reliable property-management operations.

2. Scope

This policy applies to:

- all Property Peace employees, founders, contractors, administrators, and service providers with access to company data or systems;
- the Property Peace web, mobile, API, database, cloud, messaging, document, payment, monitoring, and deployment environments;
- customer, tenant, landlord, applicant, lease, financial, authentication, and operational data;
- all development, test, staging, and production environments;
- Stripe, Azure, source-control, CI/CD, email, SMS, electronic-signature, analytics, and other third-party services used by Property Peace.

3. Governance and responsibilities

Business Owner / CEO

- Approves this policy, risk exceptions, production payment enablement, and material incident communications.
- Ensures sufficient resources exist to remediate security and financial-control risks.

Security Owner

- Maintains the security program, risk register, incident process, access reviews, vulnerability management, and annual policy review.
- Coordinates evidence collection and independent review for material releases.

Payment Operations Owner

- Owns daily payment reconciliation, disputes, refunds, negative balances, failed transfers, recovery cases, and Stripe operational alerts.
- Maintains procedures for escalating unreconciled or high-value financial events.

Engineering Owner

- Enforces secure-development, testing, review, deployment, backup, logging, and change-management controls.
- Ensures production systems match approved source and configuration.

All authorized users

- Use only assigned accounts, protect credentials, complete required security training, report suspected incidents immediately, and access only the information needed for their work.

No person may approve their own material security exception or production payment-control bypass.

4. Risk management

- Material risks must be recorded with an owner, severity, remediation plan, due date, and status.
- Critical or High risks affecting authentication, tenant isolation, money movement, secrets, or destructive data access must block release unless the Business Owner and Security Owner approve a documented, time-limited exception.
- Risk acceptance must identify compensating controls, expiration, and a review date.
- Security assessments, implementation summaries, and test results are evidence; they do not replace an approved policy or risk decision.

5. Identity and access management

- Access is granted by least privilege and business need.
- Production cloud, source-control, payment, database, and administrative access must use individually assigned accounts; shared administrative accounts are prohibited unless technically unavoidable and formally controlled.
- Multi-factor authentication is required for privileged access and must be enabled wherever supported for source control, cloud, Stripe, email, and administrative systems.
- Access must be removed promptly when a user leaves or no longer needs it.
- Privileged access must be reviewed at least quarterly; all other system access must be reviewed at least annually.
- Service accounts and API credentials must be scoped to the minimum permissions and separated by environment.
- Administrative impersonation or support access must be auditable and used only for a documented support or security purpose.

6. Secrets and configuration management

- Passwords, API keys, webhook secrets, signing keys, connection strings, tokens, and private certificates must not be committed to source control or included in tickets, documents, chat, screenshots, or logs.
- Production and development secrets must be stored in approved secret storage or protected environment configuration, preferably Azure Key Vault or an equivalent managed service.
- Development, test, and production credentials must be separate.
- Suspected or confirmed exposed credentials must be rotated immediately, affected access reviewed, and the event recorded.
- Configuration changes that affect authentication, payment intake, transfer release, data deletion, or external access require review and post-change verification.
- Secret values must be redacted from diagnostic output and evidence packages.

7. Data protection and privacy

- Property Peace will collect and retain only data needed for legitimate product, legal, security, and financial purposes.
- Sensitive and personal data must be encrypted in transit using modern TLS and encrypted at rest using approved platform controls.
- Access to tenant, landlord, applicant, lease, document, authentication, and financial data must be organization-scoped and authorized on the server.
- Production data must not be copied into development or test environments unless approved, minimized, and protected; synthetic or de-identified data is preferred.
- Data exports and document links must use controlled authorization and time-limited access where appropriate.
- Retention and deletion schedules must account for customer requests, contractual obligations, dispute needs, tax/accounting requirements, and legal holds.
- Suspected privacy incidents must follow the incident-response process and be evaluated for legal or customer-notification duties.

8. Secure software development and change management

- Server-side authorization, organization isolation, validation, and financial calculations are mandatory; frontend controls are not a security boundary.
- Material changes must be peer-reviewed or independently reviewed before production release.
- Security-sensitive and financial changes must include regression tests for authorization failures, replay, tampering, duplicate operations, concurrency, partial failure, and recovery.
- Database changes require a reviewed migration, rollback or recovery plan, and validation against the target database technology.
- Builds and tests must run against the final source being released. A pull request or commit alone is not proof of successful deployment.
- Production deployment requires traceability from approved source to deployed artifact and post-deployment verification.
- Feature flags protecting money movement or high-risk functionality must fail closed. They may be enabled only through an approved rollout plan.

- Emergency containment may be deployed before a larger redesign, but containment, redesign, migration, and production enablement must be reported and approved as separate stages.

9. Vulnerability and dependency management

- Dependencies and platforms must be scanned regularly and before material releases.
- Target remediation windows are:
 - **Critical:** contain immediately and remediate within 72 hours when feasible;
 - **High:** remediate within 14 calendar days;
 - **Medium:** remediate within 30 calendar days;
 - **Low:** remediate within 90 calendar days.
- If a deadline cannot be met, the risk owner must document exposure, compensating controls, an approved exception, and a new due date.
- Internet-facing Critical or High vulnerabilities involving payment, authentication, remote code execution, tenant isolation, or exposed secrets may require disabling the affected capability until remediated.
- Security scans must include source secrets, private keys, vulnerable packages, unsafe configuration, and relevant static-analysis checks.

10. Payment processing and financial controls

Approved architecture

- Property Peace must not store card numbers, CVCs, bank login credentials, or equivalent raw payment credentials. Stripe-hosted/tokenized methods must be used.
- Rent amount, currency, lease, tenant, organization, and destination account must be derived and validated by the server.
- Browser confirmation is not proof of final settlement and must not create final ledger accounting.
- Stripe signed webhooks and authoritative Stripe object retrieval are required for final settlement and problem-state reconciliation.
- Landlord money movement must use a platform charge followed by a separately controlled transfer unless a future architecture receives a new security and financial review.

Payment and transfer gates

- Payment intake and landlord transfer release must have independent fail-closed controls.
- A missing, invalid, or false configuration must disable the protected operation.
- Transfer release must remain disabled during incidents, unresolved reconciliation, credential uncertainty, or incomplete rollout verification.

Holds and transfer release

- Transfers must not be released before the approved method-specific hold expires.
- Current minimum policy: seven elapsed days for card payments and fourteen elapsed days for ACH/bank payments, unless a documented risk review establishes a more conservative rule.
- Immediately before transfer, the system must revalidate charge identity, PaymentIntent, amount, currency, successful state, destination, lease, tenant, and organization.
- Transfers must be idempotent and have durable state sufficient to reconcile timeouts and unknown external outcomes.

Allocation and accounting

- Rent allocation and required ledger entries must be atomic and replay-safe.
- Concurrent allocation must be serialized using database-backed controls that operate across application instances.
- Only final, authoritative settlement may reduce the amount owed.
- Processing, failed, canceled, refunded, disputed, or returned money must not be represented as fully paid unless accounting policy explicitly records and offsets the loss.

Refunds, disputes, returns, and recovery

- Refunds and disputes must record exact cumulative loss amounts and reopen the corresponding customer obligation.
- Ledger reversals must be durable, atomic, and idempotent.
- Transfer recovery must reverse only the incremental amount actually lost; it must never over-reverse the original transfer.
- Unknown Stripe outcomes must be reconciled using the same deterministic operation and idempotency key before a new operation is attempted.
- Manual refunds, transfers, reversals, or balance adjustments require documented reason, operator, amount, related payment, and post-action reconciliation.

Daily operations

The Payment Operations Owner must review and reconcile, at a frequency appropriate to active transaction volume and at least each business day when money is moving:

- Stripe platform and connected-account balances;
- successful, processing, failed, returned, and disputed payments;
- transfer-pending and reconciliation-pending records;
- reversals, recovery failures, and negative balances;
- local payment aggregates and general-ledger entries.

Material discrepancies must be escalated the same business day. Unexplained negative balances or unauthorized money movement are security incidents.

11. Logging, monitoring, and audit evidence

- Security and payment events must use structured logs with stable identifiers and timestamps.
- Logs must not contain passwords, access/refresh tokens, cookies, authorization headers, Stripe signatures, webhook secrets, connection strings, raw payment credentials, or unnecessary personal data.
- Webhook event IDs, PaymentIntent IDs, charge IDs, transfer IDs, dispute/refund IDs, state transitions, and operator actions may be logged when needed for reconciliation and appropriately protected.
- Alerts must cover authentication abuse, repeated webhook failures, prolonged processing, transfer/reversal failures, reconciliation-pending states, disputes, negative balances, and unexpected feature-gate changes.
- Logs and financial audit evidence must be access-controlled, protected from unauthorized alteration, and retained according to the approved retention schedule.

12. Incident response

Anyone who suspects unauthorized access, exposed credentials, payment loss, data disclosure, malware, destructive activity, or material control failure must notify the Security Owner and Business Owner immediately.

The incident process is:

- 1 **Identify and assess:** record time, reporter, systems, data, money movement, and known indicators.
- 2 **Contain:** disable affected features, revoke sessions, rotate credentials, restrict access, pause transfers, or isolate systems as appropriate.
- 3 **Preserve evidence:** retain relevant logs, Stripe event/object IDs, database state, deployment SHAs, configuration history, and communications without exposing secrets.
- 4 **Eradicate and recover:** remove the cause, restore trusted service, reconcile data and funds, and validate controls before re-enablement.
- 5 **Notify:** consult legal, contractual, payment-provider, insurer, and regulatory requirements; communicate with customers or partners when required and approved.
- 6 **Review:** complete a written post-incident review with root cause, impact, corrective actions, owners, deadlines, and policy updates.

No feature disabled for containment may be re-enabled solely because the immediate symptom has stopped.

13. Business continuity, backup, and disaster recovery

- Production databases and critical configuration must use automated, encrypted backups with access controls and documented retention.
- Target recovery objectives for critical customer and financial records are **RPO of four hours** and **RTO of twenty-four hours** unless a stricter contractual requirement applies.
- Backup restoration must be tested at least annually and after material storage or database changes.
- Recovery procedures must include service restoration, credential replacement, database integrity checks, Stripe/local financial reconciliation, and customer communication decision points.
- Critical vendor outages must have documented manual or degraded-mode procedures. Payment transfer release must fail closed when authoritative Stripe or database state is unavailable.
- Business continuity plans and key contacts must be reviewed annually.

14. Third-party and vendor management

- Vendors handling sensitive data, authentication, payments, communications, documents, hosting, or monitoring must be evaluated before use.
- Contracts and configurations should address security responsibilities, data use, breach notification, access control, availability, data return/deletion, and subcontractors as appropriate.
- Vendor access and API scopes must be minimized and reviewed at least annually.
- Material vendor incidents or control changes must be evaluated for Property Peace impact.

15. Security awareness and acceptable use

- Personnel with system access must receive security awareness training at onboarding and annually.
- Payment operators and developers must receive role-appropriate training on phishing, credential handling, privacy, incident reporting, and secure payment operations.

- Company systems may not be used to bypass controls, access customer data without a business purpose, or copy secrets/data into unapproved tools.

16. Exceptions and enforcement

- Exceptions require a written business reason, affected control, scope, risk, compensating controls, owner, approver, expiration date, and remediation plan.
- Exceptions affecting production payments, customer data, tenant isolation, or privileged access require approval from both the Business Owner and Security Owner.
- Violations may result in access removal, disciplinary action, contract termination, customer remediation, or legal escalation as applicable.

17. Review and approval record

This policy becomes active only when the approval fields below are completed.

- **Business Owner / CEO:** _____
- **Security Owner:** _____
- **Payment Operations Owner:** _____
- **Approval date:** _____
- **Effective date:** _____
- **Next scheduled review:** _____

Appendix A — Minimum production payment-release checklist

Before payment intake is enabled:

- ☐ Approved test and production configuration sources are identified.
- ☐ Stripe key mode and matching webhook secret are verified without displaying values.
- ☐ Exposed or plaintext credentials are removed and rotated.
- ☐ Critical and High dependency findings are remediated or covered by approved exceptions.
- ☐ Migration state matches the final released source.
- ☐ Full API and web builds/tests pass.
- ☐ Real-database concurrency and constraint evidence passes.
- ☐ Independent security review has no unresolved code-level High or Critical finding.
- ☐ End-to-end test-mode acceptance covers replay, tampering, card/ACH delay, refunds, disputes, transfer recovery, and cleanup.
- ☐ Monitoring, alert ownership, reconciliation procedure, rollback plan, and incident contacts are active.

Before transfer release is enabled:

- ☐ Payment intake has operated under observation without unresolved reconciliation issues.
- ☐ Hold policy and background processing are verified in the deployed environment.
- ☐ Transfer, reversal, ambiguous-response, and incremental-recovery tests pass.
- ☐ Negative-balance and recovery procedures are understood by the Payment Operations Owner.
- ☐ Written Business Owner and Security Owner approval is recorded.

Appendix B — Policy maintenance

Every review must consider:

- incidents, near misses, negative balances, disputes, refunds, and recovery cases;
- changes to Stripe, Azure, authentication, storage, messaging, electronic signature, or monitoring vendors;
- vulnerability and penetration-test findings;
- new laws, contracts, insurance requirements, and customer commitments;
- whether recovery objectives, backups, alerting, and daily reconciliation remain effective.

Policy limitation: This internal policy is an operational control document. It should be reviewed by qualified legal, privacy, insurance, accounting, and compliance advisers before Property Peace relies on it for contractual or regulatory claims. It does not itself establish PCI DSS or any other certification.